# Bases de Datos a Gran Escala

## Sesión 5 · Bases de datos columnares: Cassandra

Diego Sevilla Ruiz
Departamento de Ingeniería y Tecnología de Computadores
Facultad de Informática · Universidad de Murcia

dsevilla@um.es

# Bases de datos columnares

# Un caso paradigmático

BD transaccional (SQL) de una empresa. **Transacciones comerciales**, ~100 columnas:

| TrId | fecha | hora | sucursal | cuantía | cliente | precio medio ítem | nítems | pago | … |
|---|---|---|---|---|---|---|---|---|---|
| … | … | … | … | … | … | … | … | … | … |

**Cada poco tiempo**: saldo medio de las compras de cada sucursal

# Un caso paradigmático (ii)

```
SELECT sucursal, AVG(cuantía)
  FROM Transacciones
  GROUP BY sucursal;
```

**Ojo**: sólo necesitamos `cuantía` y `sucursal`

# Un caso paradigmático (iii)

Cuestiones relevantes:

1. El SGBDR almacena los elementos **por filas**
2. ⇒ un bloque de disco contiene un conjunto de filas
3. Como sólo nos interesan dos columnas, hay que desechar los otros 99 campos (lectura de bloques de disco innecesarios)

¿Y si...? Almacenamiento de los datos **por columnas** (Vertica, MonetDB):

- **Todos los datos de una columna se almacenan contiguos en disco**: se leen menos bloques
- Favorece proyecciones, agregados y compresión en consultas analíticas

Esto es **columnar analítico** (sesión 3). Cassandra y HBase pertenecen a otra familia: filas o particiones con celdas, que pueden ser dispersas, localizadas por su clave

# Introducción a Cassandra

# ¿Qué es Apache Cassandra?




- Sistema gestor de bases de datos NoSQL distribuido, de almacenamiento *wide column*, gratuito y de código abierto
- Diseñado para manejar grandes cantidades de datos en muchos servidores que no tienen que ser de alto rendimiento, con alta disponibilidad y sin un punto único de fallo
- Sólido soporte para clústeres que abarcan múltiples centros de datos, con *replicación sin maestro asíncrona* que permite operaciones de baja latencia para todos los clientes

# ¿Qué es Apache Cassandra? (ii)

- Desarrollada por dos ingenieros de Facebook para el mecanismo de búsqueda del buzón de entrada
- Más tarde, Facebook lo lanzó como proyecto de código abierto en Google Code, y después lo entregó a la fundación Apache
- Utilizada por nombres importantes como Netflix, Apple, Weather Channel, eBay y muchos más

# Uso en Netflix (por ejemplo)

## De understanding-system-design-of-netflix

**Cassandra:-**

Netflix uses Cassandra for its scalability and lack of single points of failure and for cross-regional deployments. In effect, a single global Cassandra cluster can simultaneously service applications and asynchronously replicate data across multiple geographic locations.

Cassandra data model in Netflix:-

1. 50+ Cassandra clusters
2. 500+ nodes
3. 30TB daily backups
4. 250k Writes/s at each node

# Uso en Netflix (ii)

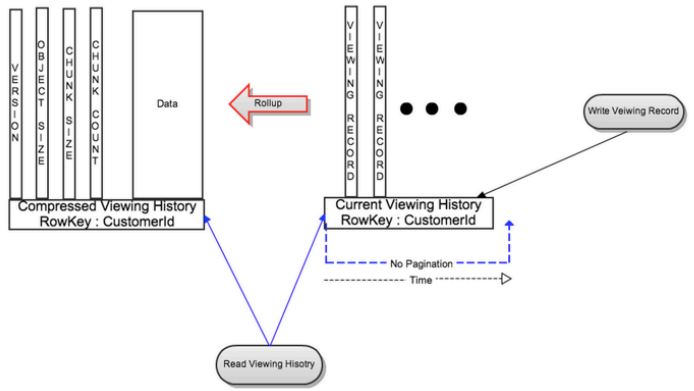
Netflix stores all kinds of data across its Cassandra DB instances, including all user-collected event metrics.

As user data began to increase, there needed to be a more efficient way to manage data storage. Netflix Redesigned data storage architecture with two main goals in mind:

- Smaller Storage Footprint.
- Consistent Read/Write Performance as viewing per member grows.

The solution to the large data problem was to compress the old rows. Data were divided into two types:

- Live Viewing History (LiveVH): Small number of recent viewing records with frequent updates. The data is stored in uncompressed form.
- Compressed Viewing History (CompressedVH): A large number of older viewing records with rare updates. The data is compressed to reduce the storage footprint. Compressed viewing history is stored in a single column per row key.




*Compressed Viewing History*

# Modelo de datos en Cassandra

- Cassandra es similar a SQL en el sentido de que almacena *tablas* y *filas*
- El lenguaje utilizado para crear tablas y manipular datos es CQL, similar a SQL
- Sin embargo, difiere de SQL en bastantes aspectos: al especificar el esquema de una tabla ( `CREATE TABLE` ), le estamos diciendo ***cómo va a distribuir los datos entre los hosts de almacenamiento***
- La especificación `PRIMARY KEY` indica en cuál de los nodos se almacenará cada entrada: por eso se llama ***clave de partición***

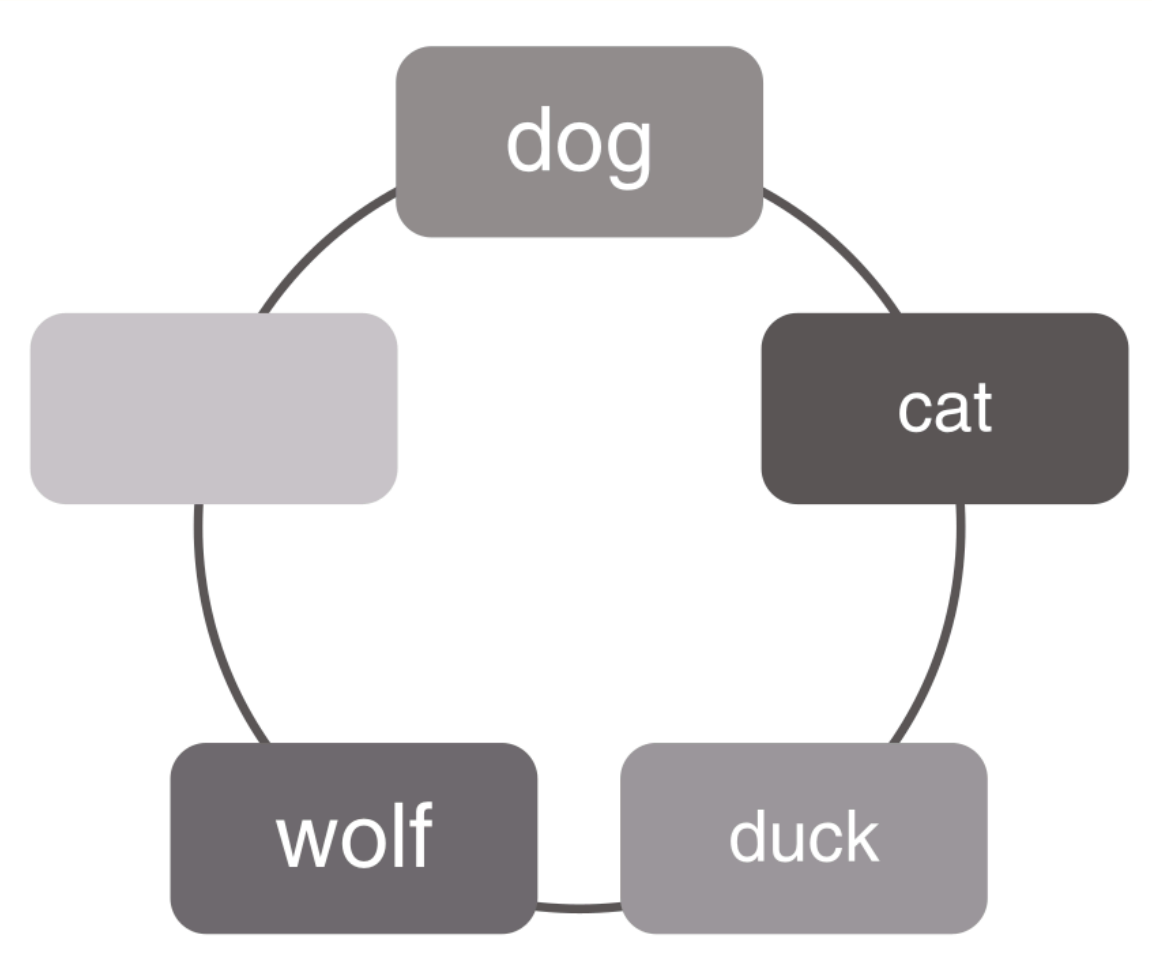
# Modelo de datos: ejemplo

```
CREATE TABLE animals (
name TEXT PRIMARY KEY,
species TEXT,
subspecies TEXT,
genus TEXT,
family TEXT);
```

`name` especifica en qué ordenador residirá cada fila:

```
select * from animals;
 name | family   | genus | species           | subspecies
------+----------+-------+-------------------+-----------------
  cat |  Felidae | Felis |         F. catus |             null
 duck | Anatidae |  Anas | A. platyrhynchos |             null
```

# Modelo de datos: claves de partición



# Claves anidadas (compuestas)

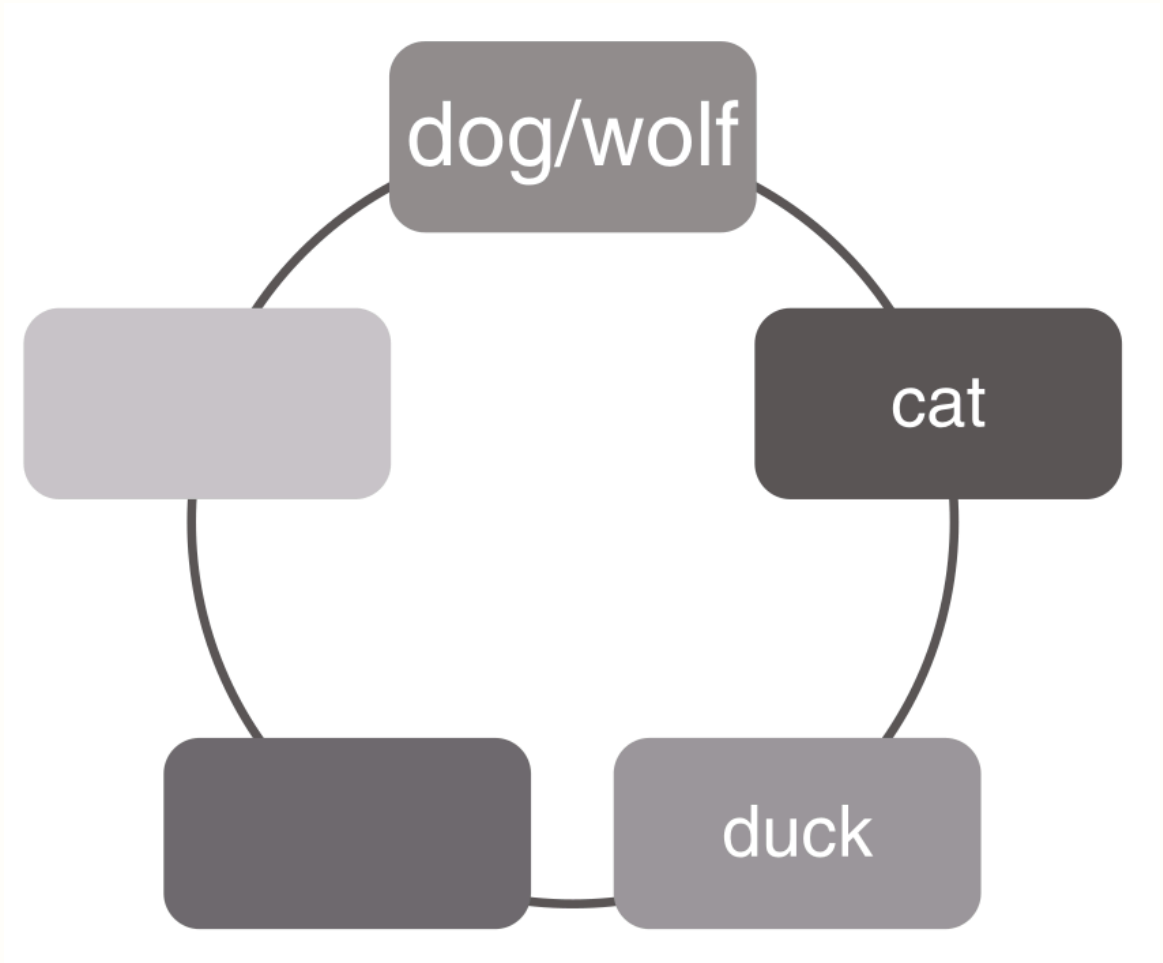
- Las claves también pueden ser ***anidadas***
- La primera parte especifica la clave de partición, y la segunda, la clave de **agregación** (cómo se agregan las filas para formar una *wide column*)
- Todas las filas con la misma clave de partición y de agregación forman, *de forma lógica*, una única fila

```
CREATE TABLE animals (
name TEXT,
species TEXT,
subspecies TEXT,
genus TEXT,
family TEXT,
PRIMARY KEY ((family, genus), name)
);
```

# Claves compuestas: resultado

```
select * from animals;
 family   | genus | name | species           | subspecies
----------+-------+------+-------------------+-----------------
 Anatidae |  Anas | duck | A. platyrhynchos  |            null
  Felidae | Felis |  cat |         F. catus  |            null
  Canidae | Canis |  dog |         C. lupus  | C. l. familiaris
  Canidae | Canis | wolf |         C. lupus  |            null
```

# Claves compuestas



# Modelo de datos vs. SQL

Sistema que recoge eventos, en SQL:

```
CREATE TABLE events (
id INT PRIMARY KEY,
time TIME,
event_type INT references event_types(id),
data text
);
```

En Cassandra no hay claves ajenas: referenciar el tipo de evento por su `id` no es posible

# Modelo de datos vs. SQL (ii)

Lo que se suele hacer en NoSQL es ***desnormalizar***: incluir la descripción del tipo de evento en la propia tabla:

```
CREATE TABLE events (
id UUID PRIMARY KEY,
time TIMESTAMP,
event_type TEXT,
data text
);
```

Esto funciona y es una copia de cómo lo haríamos en SQL

# Modelo de datos vs. SQL (iii)

- Sin embargo, en Cassandra **TAMPOCO** podemos filtrar filas que no sean la clave primaria: Cassandra considera que tardaría demasiado (y de hecho es así)
- Sería difícil encontrar eventos por otras claves que no sean su `id`, como la hora o el tipo de evento
- ¿Cómo solucionarlo?

# Modelo de datos vs. SQL (iv)

- **Opción 1**: indexar el campo `time`
  - Puede generar una alta carga, ya que los eventos estarán dispersos en el clúster
- **Opción 2**: utilizar el campo `time` como clave de partición (`PRIMARY KEY`)
  - En una clave primaria con múltiples campos, la primera actúa como clave de partición
  - Los campos adicionales forman parte del nombre de la columna (una *wide column* o *wide row*)

# Modelo de datos vs. SQL (v)

Veamos la opción 2:

```
CREATE TABLE events (
hour TIMESTAMP,
id UUID,
time TIMESTAMP,
event_type TEXT,
data text,
PRIMARY KEY(hour, id)
);
```

Con este modelo podemos consultar fácilmente todos los eventos de una misma hora: todo lo ocurrido en esa hora está en una única fila en el disco

# Modelo de datos vs. SQL (vi)

- Si se tienen lecturas y/o escrituras intensas, puede ser necesario segmentar la fila en varias, con una granularidad más fina
- Un evento por fila es difícil de consultar; una fila por hora puede generar **cuellos de botella** y filas muy grandes (todas las lecturas/escrituras irían a los mismos archivos en disco)
- Podemos segmentar en varias filas fáciles de leer, siempre que tengamos algo por lo que consultar: aquí, `event_type`
- Podemos mejorar el rendimiento asegurando que el orden se guarda por la hora del evento, no por el `id`

# Modelo de datos vs. SQL (vii)

```
CREATE TABLE events (
hour TIMESTAMP,
id UUID,
time TIMESTAMP,
event_type TEXT,
data text,
PRIMARY KEY((hour, event_type), time)
) WITH CLUSTERING ORDER BY (time DESC);
```

# Modelo de datos vs. SQL (viii)

```
SELECT * FROM events;
hour                | event_type | time                | data      | id
--------------------+------------+---------------------+-----------+-------
2013-06-13 11:00:00 | click      | 2013-06-13 11:00:00 | some data | 3...
2013-06-13 11:00:00 | page_view  | 2013-06-13 11:00:01 | some data | 2...
2013-06-13 11:00:00 | error      | 2013-06-13 11:00:05 | some data | 0...
2013-06-13 11:00:00 | redirect   | 2013-06-13 11:00:09 | some data | 1...
```

# Colecciones

- Cassandra incluye colecciones como parte de su modelo de datos
- Pueden proporcionar flexibilidad en las consultas: **conjuntos**, **listas** y **mapas**

**Conjuntos** ( `SET` ): mantienen un conjunto único de elementos sin necesidad de leer antes de escribir

```
CREATE TABLE users (
email TEXT PRIMARY KEY,
portfolios SET<UUID>,
tickers SET<TEXT>
);
```

# Conjuntos: actualización

```
UPDATE users
  SET portfolios = portfolios + {756716f7-2e54-4715-9f00-91dcbea6cf50},
      tickers = tickers + {'AMZN'}
WHERE email = 'foo@bar.com';
UPDATE users
  SET tickers = tickers + {'GOOG'}
WHERE email = 'foo@bar.com';

email         | portfolios                             | tickers
foo@bar.com | {756716f7-2e54-4715-9f00-91dcbea6cf50} | {'AMZN', 'GOOG'}
```

# Conjuntos: eliminar elementos y borrar

```
UPDATE users
SET tickers = tickers - {'AMZN'}
WHERE email =  'foo@bar.com';
-- tickers => {'GOOG'}

DELETE tickers
FROM users
WHERE email = 'foo@bar.com';
-- tickers => null
```

# Listas

Cuando no se requiere unicidad y se necesita mantener el orden. Ejemplo: permitir que los usuarios especifiquen sus cinco *tickers* principales:

```
ALTER TABLE users ADD top_tickers list<text>;

UPDATE users SET top_tickers = ['GOOG'] WHERE email = 'foo@bar.com';
UPDATE users SET top_tickers = top_tickers + ['AMZN'] WHERE email = 'foo@bar.com';

-- top_tickers => ['GOOG', 'AMZN']
```

# Mapas

Objeto tipo diccionario, útil para datos tabulares dentro de una fila (ayuda a mitigar la falta de `JOIN` s). Ejemplo: fecha de actualización por *ticker*:

```
ALTER TABLE users ADD ticker_updates map<text, timestamp>;

UPDATE users SET ticker_updates = { 'AMZN':'2013-06-13 11:42:12' }
WHERE email = 'foo@bar.com';

UPDATE users SET ticker_updates['GOOG'] = '2013-06-13 12:51:31'
WHERE email = 'foo@bar.com';
```

# Mapas: borrado

```
DELETE ticker_updates['AMZN']
FROM users
WHERE email = 'foo@bar.com';

email         | portfolios     | ticker_updates
foo@bar.com | {756716f7...} | {'GOOG': '2013-06-13 12:51:31-0400'}
```

# Distribución de datos en Apache Cassandra

- Para distribuir los datos, Cassandra usa un modelo *peer-to-peer* basado en claves de partición
- Para determinar qué nodo contiene un dato, Cassandra usa ***tokens***
- Un token es un entero de 64 bits; cada nodo del clúster tiene asignado un rango de tokens
- Al buscar una fila, Cassandra calcula el token asociado (con una función *hash*) y determina de forma única el nodo al que pertenece
- Cassandra usa así un conjunto de tablas *hash* distribuidas
- Al añadir o eliminar nodos, hay que redistribuir los tokens asignados

> ¿Qué son las tablas *hash* distribuidas (THD)?

# Tablas *hash* distribuidas

- Una base de datos distribuida que almacena y recupera información sobre claves en nodos que pueden unirse o abandonar en cualquier momento
- Los nodos se coordinan sólo entre sí, sin ningún nodo central coordinador
- Arquitectura con nodos réplica ⇒ tolerante a fallos
- Las claves se mantienen, en general, distribuidas uniformemente entre los nodos
- Para almacenar en múltiples nodos, se toma el *hash* módulo número de servidores
- Si uno de los nodos falla, hay que cambiar el módulo con el nuevo número de servidores, y empezamos a perder información

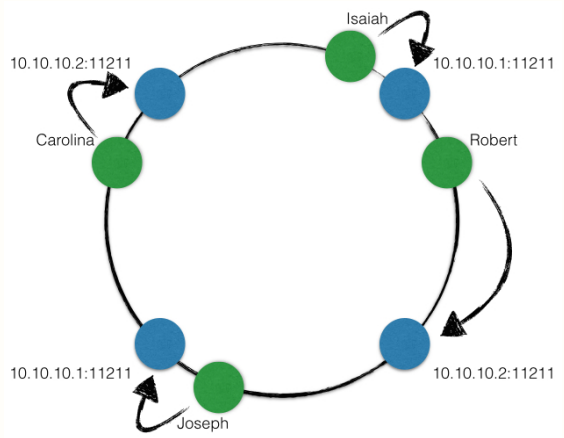
> ¿Recordamos qué era el *Consistent Hashing* (CH)?

# *Consistent Hashing*

- Similar a lo que pasaba en MySQL con `PARTITION`: las tablas se distribuyen entre distintos *hosts*
- A cada clave se le calcula un *hash*; los posibles valores se disponen en un círculo con aritmética módulo $K$
- A cada *host* $h_i$ del anillo se le calcula también un valor de *hash*
- Una fila con clave $k$ se almacena en el *host* $h_i$ que cumple:

$$hash(h_{i-1}) <_{mod\ K} hash(k) <_{mod\ K} hash(h_i)$$

(se almacena en el siguiente *host* siguiendo el círculo)



# *Consistent Hashing* (ii)

Para que las claves se distribuyan uniformemente entre servidores:

- Se asignan no una, sino muchas etiquetas (*angles*) a cada servidor: en vez de `A`, `B` y `C`, tener `A0` … `A9`, `B0` … `B9` y `C0` … `C9`, entremezcladas en el círculo
- El factor por el que aumentar el número de etiquetas (*server keys*, o peso) depende de la situación, y puede ser diferente para cada servidor
- Si el servidor `B` fuera el doble de potente, se le asignaría el doble de etiquetas, y terminaría manteniendo el doble de objetos en promedio

# *Consistent Hashing* (iii)

Si eliminamos un servidor ( `C` )...

- Hay que eliminar las etiquetas `C0` ... `C9` del círculo
- Las claves antes adyacentes a esas etiquetas se reasignan aleatoriamente a `A` y `B` ; el resto de claves queda intacto

Si añadimos un servidor ( `D` )...

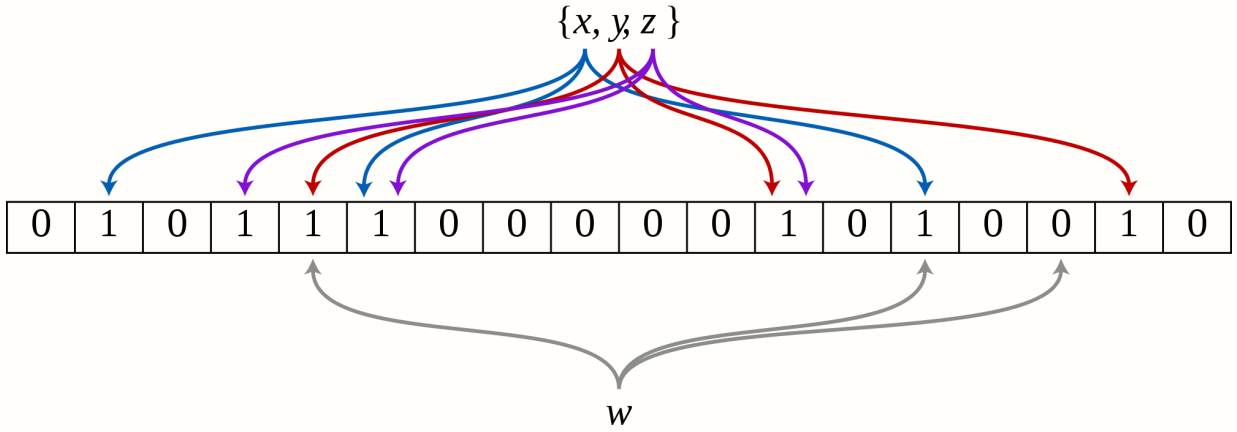
- Hay que agregar las etiquetas `D0` ... `D9`
- Aproximadamente un tercio de las claves existentes (de `A` o `B` ) se reasignan a `D` ; el resto permanece igual

# Inciso: *Bloom Filters*

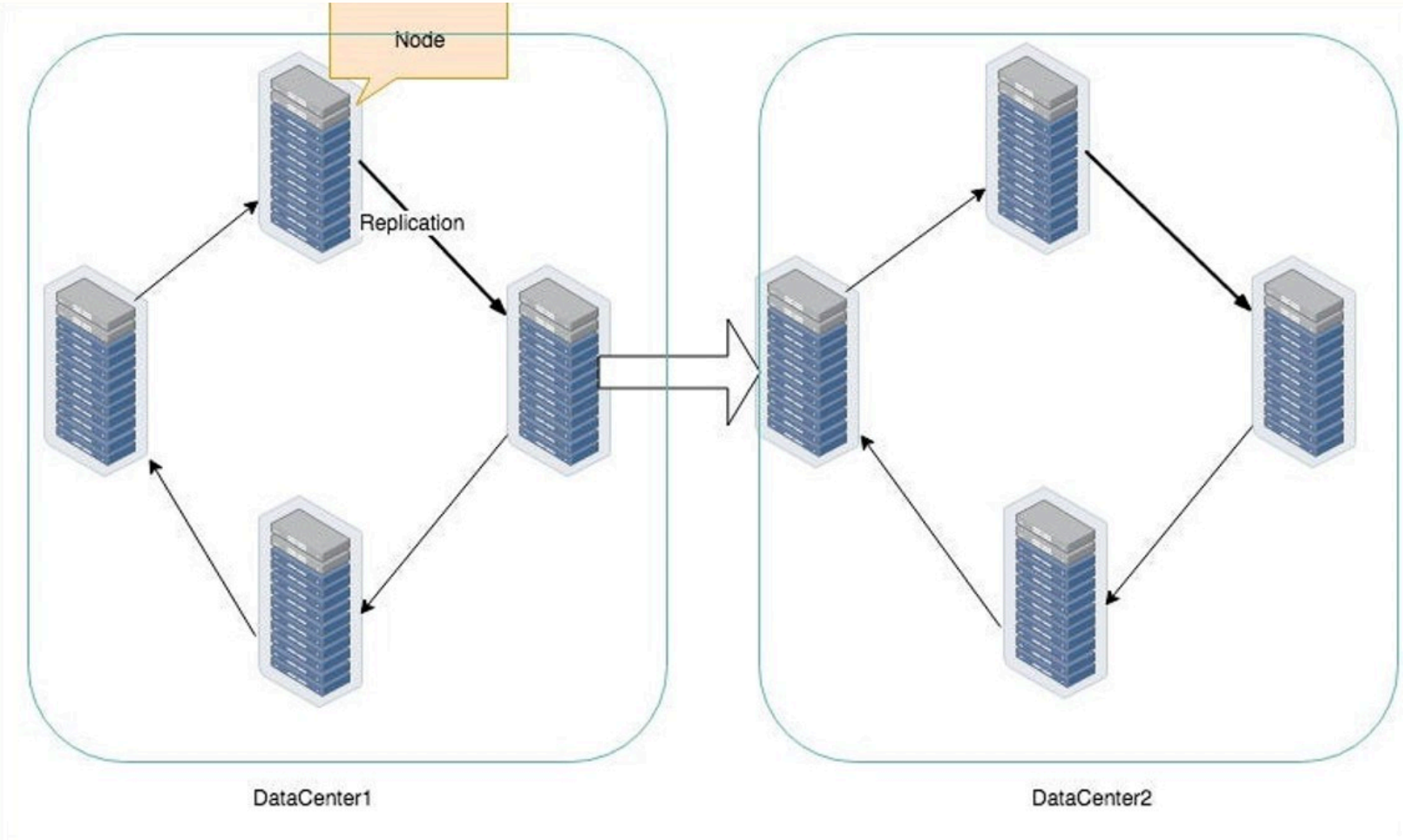
- Otra tecnología habilitadora muy famosa en sistemas distribuidos
- Permiten descartar muy rápido si un elemento está en un conjunto:
  - Si la respuesta es **NO**, el elemento no está en el conjunto
  - En otro caso, no se sabe nada (hay que buscar dentro del conjunto)
- Permiten descartar *hosts* enteros: se sabrá, por ejemplo, que una columna no está en ninguna fila de ese *host*

# *Bloom Filters*: funcionamiento

- Se construye un mapa de $k$ bits, donde $k$ es el número de bits de una función *hash*
- Cada elemento $e$ insertado actualiza el mapa: $mapa = mapa \mid hash(e)$ (se añaden los bits del *hash*)
- Un elemento $e'$ ***puede estar*** en el mapa si $hash(e') = mapa \;\&\; hash(e')$ (están todos los bits del *hash*)



# Componentes principales



# Componentes principales: estructurales

- **Nodo**: componente básico de los datos, máquina donde se almacenan
- **Centro de datos (*DataCenter*)**: colección de nodos relacionados; puede ser físico o virtual
- **Clúster**: contiene uno o más centros de datos, y puede extenderse a través de ubicaciones

# Componentes principales: internos

- ***Commit Log***: cada escritura se almacena primero aquí; se usa para la recuperación ante fallo
- ***Mem-Table***: tras escribirse en el *commit log*, los datos se almacenan aquí hasta alcanzar un umbral
- ***SSTable***: archivo en disco que almacena los datos de la *MemTable* al alcanzar el umbral; se almacenan secuencialmente y se mantienen por tabla

# Funcionamiento general

- Cada nodo puede realizar tanto lecturas como escrituras
- No existe jerarquía con nodos maestros o esclavos dentro de un clúster
- Cassandra usa el protocolo *Gossip* para mantener el clúster:
  - Protocolo de comunicación entre pares en el que los nodos intercambian periódicamente información sobre su estado y el de otros nodos
  - Se ejecuta cada segundo, intercambiando mensajes de estado

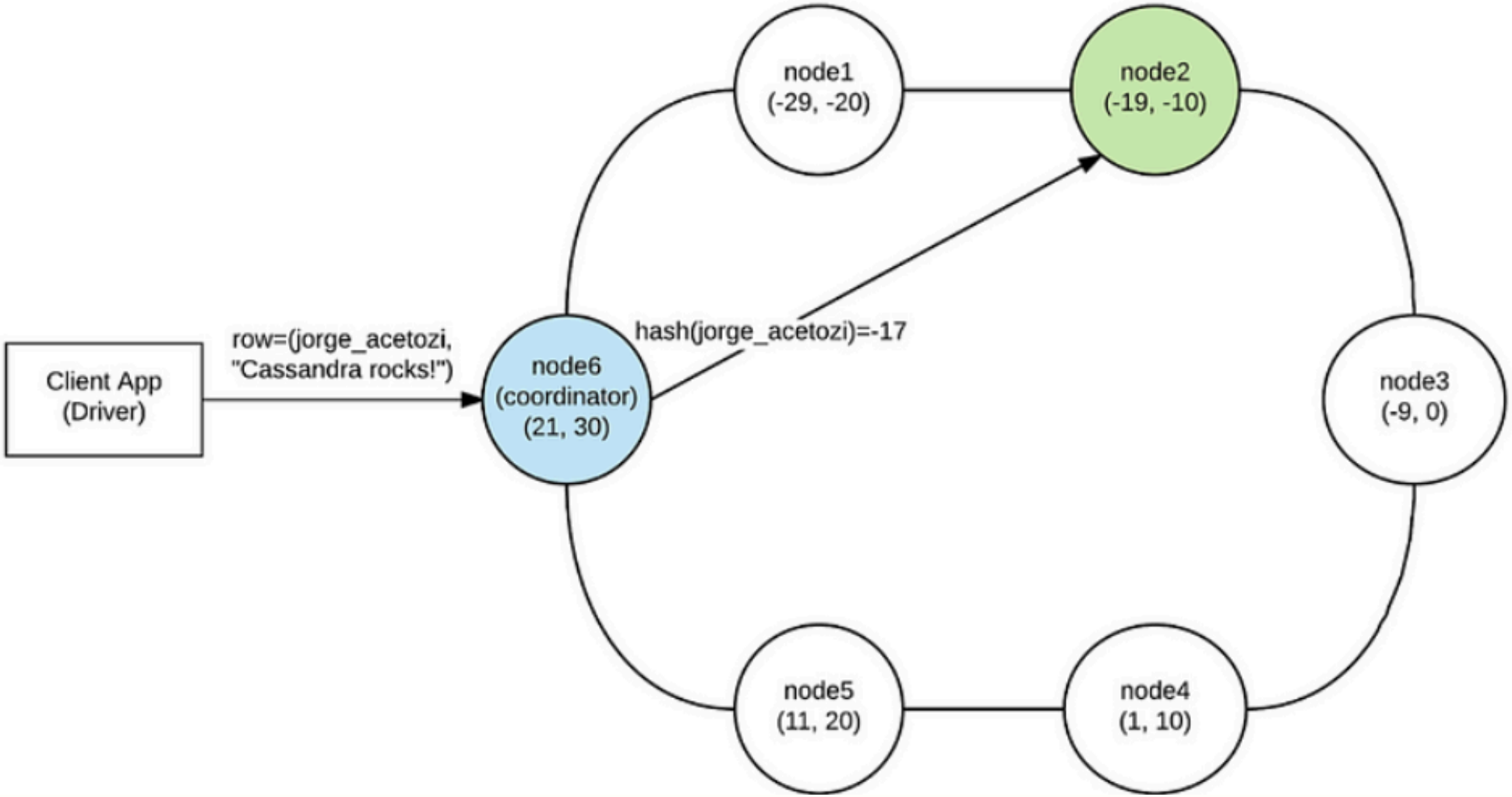
# Funcionamiento general (ii)

- Cuando un cliente envía una solicitud a un clúster Cassandra, llega primero al nodo ***coordinador***, que actúa como proxy entre el cliente y los nodos involucrados
- El *partitioner* es el componente responsable de determinar cómo distribuir los datos entre los nodos: calcula el *hash* de la clave de partición de una fila
- Según la política de partición de datos (no se ve en clase), un mismo nodo podría actuar o no, al mismo tiempo, como coordinador y *partitioner*

# Ejemplo de nodo coordinador

Un cliente quiere escribir una nueva fila cuya clave de partición es `jorge_acetozi`. El nodo 6 (coordinador) calcula que el *hash* de dicha clave toma el valor `-17`

Por el mecanismo definido por el *partitioner* (no se muestra en la figura), esa fila debe almacenarse en el nodo 2 del clúster

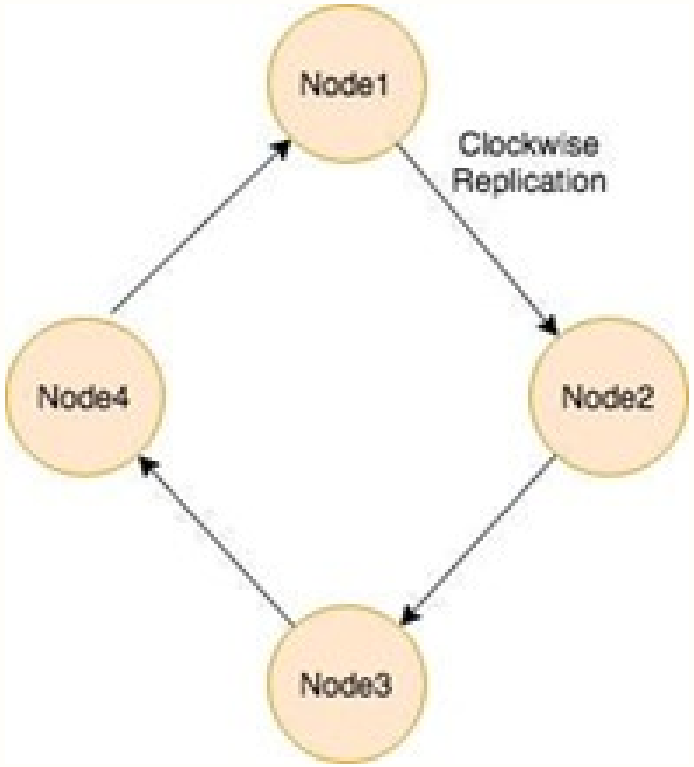

node1
(-29, -20)
node2
(-19, -10)
Client App
(Driver)
row=(jorge_acetozi,
"Cassandra rocks!")
node6
(coordinator)
(21, 30)
hash(jorge_acetozi)=-17
node3
(-9, 0)
node5
(11, 20)
node4
(1, 10)

# Consistencia en Apache Cassandra

- Cassandra permite ajustar disponibilidad y consistencia configurando *replication factor* y *consistency level*
- El nivel de consistencia se define por consulta, y permite ajustar cuándo se ofrece el resultado a los clientes
- El factor de replicación asegura que las escrituras se envíen a todas las réplicas
- Ejemplo: con consistencia 3 en un clúster con 3 nodos, se necesitaría que los 3 nodos tengan consenso para tener el dato disponible

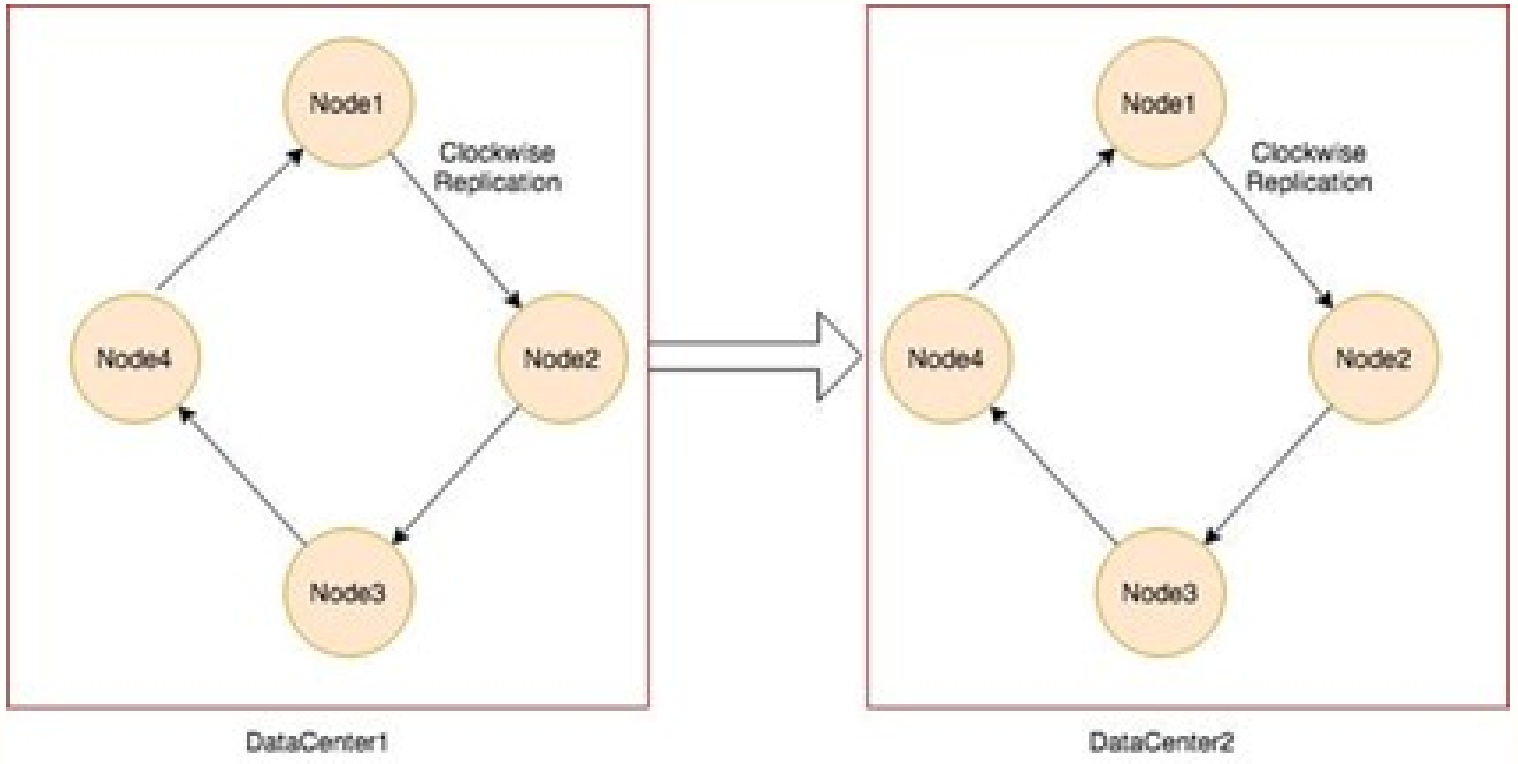
# Estrategias de replicación: `SimpleStrategy`

Despliegues en un solo centro de datos; no considera la topología de la red. Sólo toma la decisión del *partitioner* y coloca las réplicas restantes en sentido horario



# Estrategias de replicación: `NetworkTopologyStrategy`

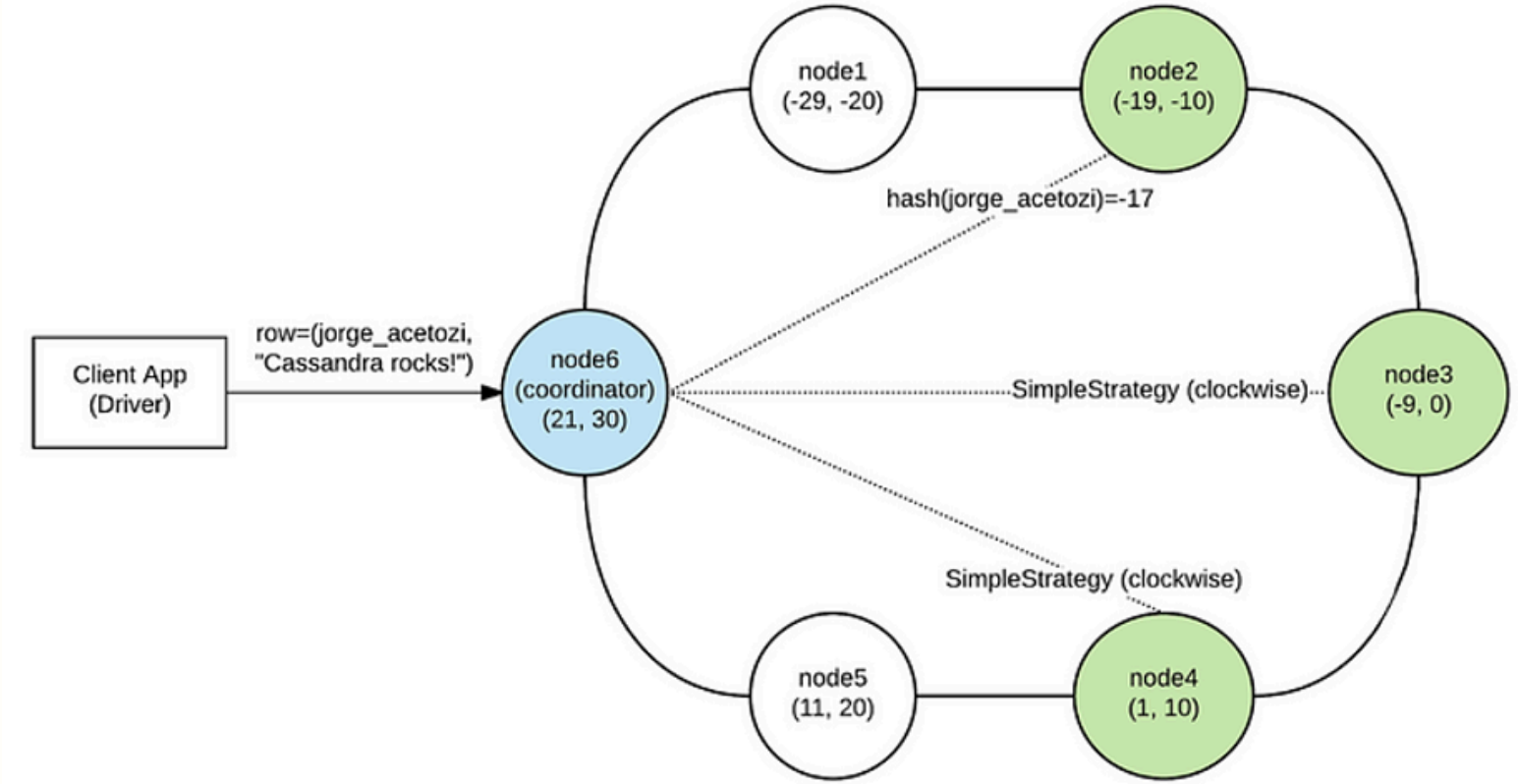
Despliegues en múltiples centros de datos. También parte de la decisión del *partitioner*, pero tiene en cuenta la configuración de *racks* y *datacenters*



# Niveles de consistencia de escritura

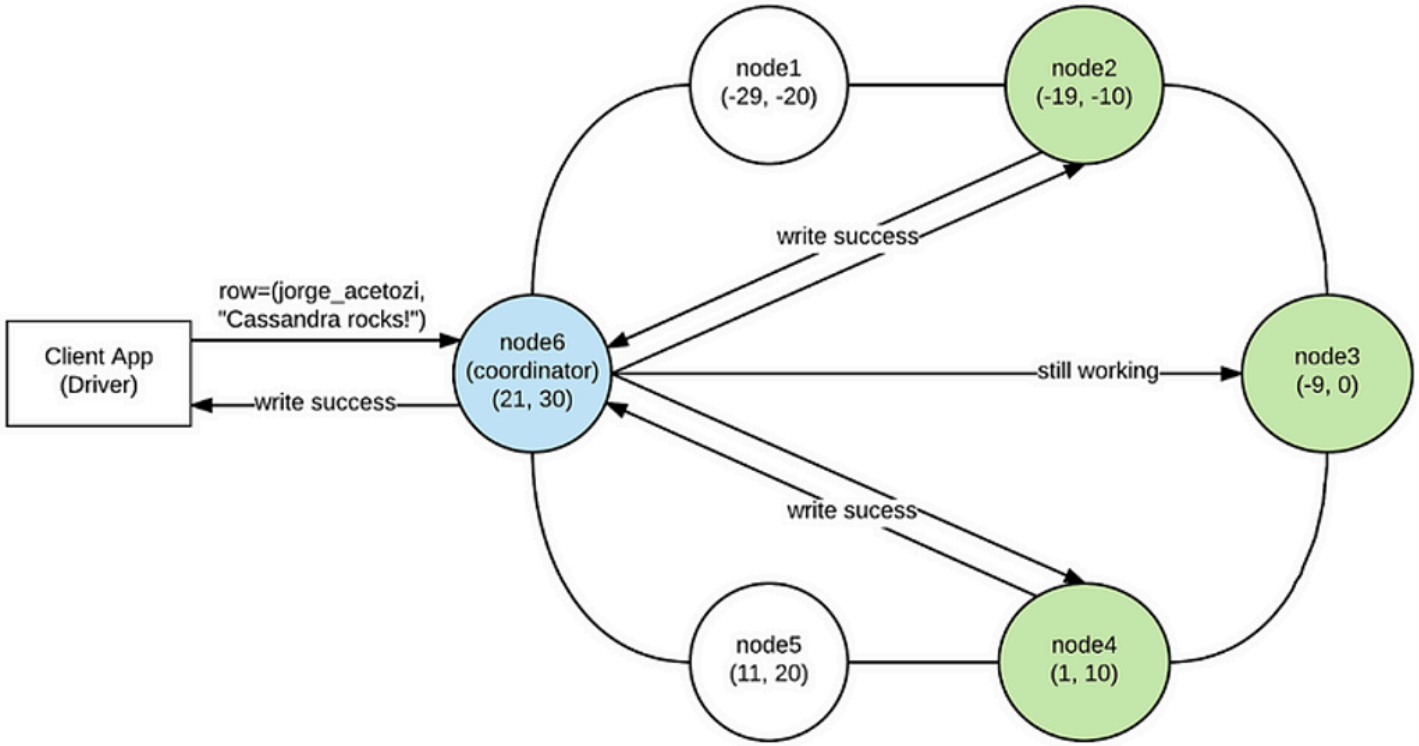
- Al insertar datos se puede especificar el nivel de consistencia de escritura (`USING CONSISTENCY`, o dentro de un perfil de ejecución — se ve más adelante)
- Es el número de réplicas que deben confirmar al coordinador que su inserción local fue exitosa
- En cuanto recibe tantos *acknowledgements* como el nivel configurado, el coordinador informa al cliente del éxito, sin esperar a las réplicas restantes
- Niveles: `ANY`, `ONE`, `TWO`, `THREE`, `QUORUM`, `LOCAL_QUORUM`, `EACH_QUORUM`, `ALL`
- `ONE` es aceptable si se necesita rendimiento muy alto y no importa perder alguna escritura; `QUORUM` asegura que la mayoría de nodos responda antes de considerar la operación exitosa

# Escritura con tres réplicas: el coordinador reparte la fila



# Estrategias de replicación: ejemplo

Escenario con `WRITE_CONSISTENCY_LEVEL=2` y 3 nodos réplica configurados. En cuanto llega la respuesta de los nodos 2 y 4, el coordinador informa al cliente de que la escritura se realizó con éxito



# Niveles de consistencia y sus compromisos

Cada lectura y cada escritura eligen cuántas réplicas deben responder. Con factor de replicación (RF) $n$:

| Nivel | Réplicas que responden | Compromiso |
|---|---|---|
| `ANY` (sólo escritura) | ninguna en concreto: vale una *pista* (*hint*) guardada por el coordinador | la escritura más rápida y disponible, pero puede tardar en ser visible |
| `ONE` / `LOCAL_ONE` | 1 | rápido y muy disponible, pero puede leer una réplica atrasada |
| `QUORUM` / `LOCAL_QUORUM` | la mayoría: $\lfloor n/2 \rfloor + 1$ | equilibrio: consistencia sin exigir que estén todas vivas |
| `ALL` | las $n$ | la máxima consistencia, pero falla en cuanto cae un solo nodo |

`LOCAL_` cuenta sólo las réplicas del *datacenter* local; con un único centro de datos equivale al nivel sin prefijo

# Escribir a `ONE` no espera a las demás réplicas

- El coordinador envía la escritura a **todas** las réplicas a la vez, pero responde en cuanto contestan las que exige el nivel
- Con `ONE`, las demás la aplican un instante **después**: una lectura inmediata a `ONE` puede caer en una réplica atrasada y devolver el dato antiguo
- Sólo una lectura de **varias** réplicas *reconcilia*: el coordinador pide los datos a una réplica y un resumen (*digest*) a las demás; si difieren, se queda con la versión de marca de tiempo más reciente y repara a las atrasadas (*read repair*)
- La regla que garantiza ver la última escritura: $R + W > RF$, donde $R$ y $W$ son las réplicas leídas y escritas, porque así comparten al menos una

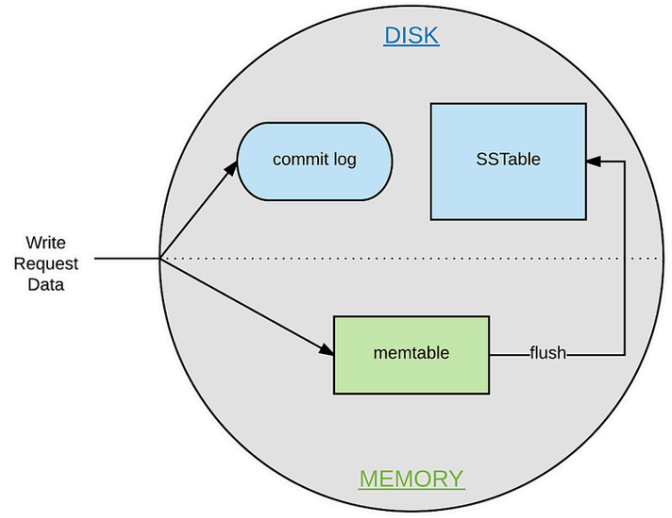
# Por qué `LOCAL_QUORUM` por omisión

- Con dos nodos (RF = 2) la mayoría es 2, luego $R + W = 2 + 2 > 2$: cada lectura ve lo último escrito, con independencia de qué réplica responda antes
- El precio es la **disponibilidad**: con RF = 2 necesita los dos nodos, y si uno cae falla con `Unavailable`, mientras que `ONE` seguiría respondiendo
- En producción se usa RF = 3: la mayoría (2) sigue disponible con un nodo caído
- Con un solo nodo (RF = 1) se comporta como `ONE`
- La carga masiva usa `ANY`, porque sólo importa la velocidad; las lecturas posteriores, con `LOCAL_QUORUM`, consultan las réplicas y reconcilian

# Procesamiento de las escrituras

Cuando llega una solicitud de escritura a un nodo, ocurren principalmente dos cosas:

- Se agrega al ***commit log*** en disco: garantiza la durabilidad (sobrevive incluso ante un fallo del nodo)
- Se envía a la ***memtable***: cuando se llena, los datos se vuelcan a una ***SSTable*** en disco, y el *commit log* se limpia



# Procesamiento de las lecturas

- Los filtros de Bloom identifican aproximadamente si una fila/columna existe en una SSTable
- Se mantienen por SSTable: cada una tiene un filtro de Bloom correspondiente en memoria

> Un filtro de Bloom es una estructura de datos genérica para verificar si un elemento está presente en un conjunto o no

- Si la respuesta es «puede estar presente», Cassandra busca en la SSTable; si no está, continúa en otras SSTables
- Así ahorra muchas exploraciones innecesarias, y un gran coste de E/S de disco

# Creación de índices

- En Cassandra los índices secundarios son complicados de usar y pueden afectar mucho al rendimiento
- La tabla de índices se almacena en cada nodo del clúster: una consulta con un índice secundario puede convertirse en una pesadilla de rendimiento si accede a varios nodos
- Regla general: indexar una columna con **baja cardinalidad** de pocos valores
- Actualmente este tipo de índices sufre de bajo rendimiento y alta latencia

# SAI (*Storage-Attached Index*)

- Nuevo tipo de índice en Apache Cassandra (desde 4.0), con mejor rendimiento que los índices secundarios tradicionales
- Permite búsquedas eficientes en columnas no clave
- Soporta igualdad, rangos y búsquedas parciales en texto

**Ventajas**:

- Menor consumo de almacenamiento
- Soporte para múltiples tipos de datos
- Consultas más eficientes que con índices tradicionales

**Consideraciones**:

- No recomendado para columnas con alta cardinalidad
- Puede afectar el rendimiento en inserciones frecuentes

# Índice SAI vs. clave de partición

| Característica | Índice SAI | Clave de partición |
|---|---|---|
| Función principal | Búsquedas en columnas no clave | Distribución de datos en nodos |
| Impacto en almacenamiento | Crea estructuras adicionales | Define ubicación en el clúster |
| Rendimiento en consultas | Optimiza búsquedas secundarias | Muy rápido en clave de partición |
| Eficiencia en escrituras | Puede ralentizar inserciones | Altamente eficiente |
| Uso recomendado | Igualdad, rango y texto | Consultas rápidas y directas |

# Acceso a la base de datos

- Cassandra usa CQL (*Cassandra Query Language*) para interactuar con la base de datos
- Apache Cassandra cuenta con una *shell* de línea de comandos, `cqlsh`, sobre la cual se pueden invocar las consultas CQL
- Para el acceso desde Python contamos con la librería `cassandra-driver` (`pip install cassandra-driver`)
- Al conectarnos a un clúster, especificamos distintos **perfiles de ejecución** (`ExecutionProfile`), que permiten ejecutar consultas de forma diferente en una misma sesión

# Conexión al clúster: puntos de contacto y perfiles

Conectarse a Cassandra no se parece a abrir una conexión a un servidor SQL: el *driver* mantiene un *pool* de conexiones **con todos los nodos que considera accesibles**, y decide, consulta a consulta, a cuál se la envía

- **Puntos de contacto** (`contact_points`): sólo se usan en el arranque. El *driver* abre la *control connection* contra uno de ellos y descubre el resto por *gossip* (`system.local` / `system.peers`)
- **Hosts** (`cluster.metadata.all_hosts()`): cada nodo descubierto, con dirección, *datacenter*, *rack* y estado. El *driver* se conecta a la dirección que **cada nodo publica** (`rpc_address` / `broadcast_rpc_address`), no a la de `contact_points`
- **Perfiles de ejecución** (`ExecutionProfile`): agrupan política de balanceo, consistencia, reintentos, *timeout* y `row_factory`; cada consulta elige uno con `execution_profile=`

# Modelo antiguo frente a perfiles

```
cluster = Cluster(["cassandra1"], load_balancing_policy=...)   # obsoleto
session.default_timeout = 40                                   # obsoleto
session.row_factory = dict_factory                              # obsoleto
```

Los dos modelos son **excluyentes**: en cuanto se pasa `execution_profiles=` al `Cluster`, cualquier atributo del modelo antiguo lanza un error:

- `Cluster(..., load_balancing_policy=..., execution_profiles=...)` → `ValueError`
- `session.default_timeout = 40` → `ValueError: usar un perfil en su lugar`

El *timeout* pasa a ser `request_timeout` **dentro** del perfil; lo mismo para `row_factory` y el nivel de consistencia por omisión

# Políticas de balanceo

- `RoundRobinPolicy`: reparte entre todos los nodos vivos
- `DCAwareRoundRobinPolicy(local_dc=...)`: sólo usa los nodos del *datacenter* local. Si `local_dc` no coincide **exactamente** con el nombre real, todos los nodos se consideran remotos y, con `used_hosts_per_remote_dc=0` (valor por omisión), el *driver* se queda sin destinos: `NoHostAvailable`
- `TokenAwarePolicy(política_hija)`: con sentencias preparadas, envía la petición directamente a una **réplica** de la partición, ahorrando un salto de coordinación
- `WhiteListRoundRobinPolicy` / `HostFilterPolicy`: restringen el conjunto de nodos utilizables — útiles para fijar consultas a un nodo (administración, depuración), **no** para producción: anulan el reparto de carga y el *token awareness*

# Crear los perfiles y conectar

```
from cassandra import ConsistencyLevel
from cassandra.cluster import EXEC_PROFILE_DEFAULT, Cluster, ExecutionProfile
from cassandra.policies import RoundRobinPolicy, TokenAwarePolicy
from cassandra.query import named_tuple_factory

query_profile = ExecutionProfile(   # perfil por omisión: consultas interactivas
    load_balancing_policy=TokenAwarePolicy(RoundRobinPolicy()),
    consistency_level=ConsistencyLevel.LOCAL_QUORUM,  # mayoría de réplicas; lee y reconcilia
    request_timeout=40.0,
    row_factory=named_tuple_factory,
)
load_profile = ExecutionProfile(    # carga masiva: ANY sólo existe para escritura
    load_balancing_policy=TokenAwarePolicy(RoundRobinPolicy()),
    consistency_level=ConsistencyLevel.ANY,
    request_timeout=120.0,
    row_factory=named_tuple_factory,
)
cluster = Cluster(
    contact_points,
    execution_profiles={EXEC_PROFILE_DEFAULT: query_profile, "load": load_profile},
)
session = cluster.connect()
```

# Un perfil de ejecución por nodo

Se añaden **después** de conectar, con `cluster.add_execution_profile()`. La lista blanca se construye con `host.address` —la dirección que publica el nodo—, no con el nombre del servicio Docker:

```
from cassandra.policies import WhiteListRoundRobinPolicy

for numero, host in enumerate(cluster.metadata.all_hosts(), start=1):
    cluster.add_execution_profile(
        f"node{numero}",
        ExecutionProfile(
            load_balancing_policy=WhiteListRoundRobinPolicy([host.address]),
            consistency_level=ConsistencyLevel.ONE,
            request_timeout=40.0,
            row_factory=named_tuple_factory,
        ),
    )
```

Forma correcta de dirigir una consulta a un nodo concreto: no hay un parámetro «ejecuta esto aquí»

# Perfil consciente del *datacenter*

`DCAwareRoundRobinPolicy` es la política habitual en producción, junto con `LOCAL_ONE` / `LOCAL_QUORUM`: las consultas no cruzan la WAN. El nombre del DC **no** se codifica a mano — se toma de los metadatos ya descubiertos:

```
from cassandra.policies import DCAwareRoundRobinPolicy

local_dc = cluster.metadata.all_hosts()[0].datacenter  # no hardcodear el nombre

cluster.add_execution_profile(
    "local_dc",
    ExecutionProfile(
        load_balancing_policy=TokenAwarePolicy(DCAwareRoundRobinPolicy(local_dc=local_dc)),
        consistency_level=ConsistencyLevel.LOCAL_ONE,
        request_timeout=40.0,
        row_factory=named_tuple_factory,
    ),
)
```

# Creación de `keyspaces`

```
session.execute("""
    CREATE KEYSPACE IF NOT EXISTS mi_keyspace
    WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1}
""")
```

Genera un *keyspace* con `SimpleStrategy`, replicando en un único nodo: válido para un clúster mínimo de desarrollo, pero **ignora la topología** del clúster

# `keyspaces` en un clúster con varios DC

Con `GossipingPropertyFileSnitch`, cada nodo declara su *datacenter*: la estrategia adecuada es `NetworkTopologyStrategy`, con un factor de replicación **por DC** calculado a partir de los nodos ya descubiertos:

```
from collections import Counter

replicas_por_dc = Counter(host.datacenter for host in cluster.metadata.all_hosts())
factores = ", ".join(f"'{dc}': {min(n, 3)}" for dc, n in sorted(replicas_por_dc.items()))

session.execute(f"""
    CREATE KEYSPACE IF NOT EXISTS stackoverflow
    WITH replication = {{'class': 'NetworkTopologyStrategy', {factores}}}
""")
```

`SimpleStrategy` queda desaconsejada en cuanto hay más de un *datacenter*

# Creación de tablas

```
session.execute("""
    CREATE TABLE IF NOT EXISTS mi_keyspace.usuarios (
        nombre TEXT,
        apellidos TEXT,
        edad INT,
        dni TEXT,
        PRIMARY KEY (dni, apellidos)
    )""")
```

Tabla `usuarios` dentro de `mi_keyspace`, con `dni` como clave de partición y `apellidos` como clave de agregación

# Inserción de datos

```
session.execute("""
    INSERT INTO usuarios (nombre, apellidos, edad, dni)
    VALUES ('Juan', 'Perez', 30, '11111111');
""")
```

# Consultas de datos

```
select_query = session.prepare("SELECT * FROM mi_keyspace.usuarios")
rows = session.execute(select_query)
for row in rows:
    print(row)
```

Filtrar por la clave de partición ( `dni` ) es directo y eficiente, y **siempre** debe usarse una sentencia preparada (evita inyección CQL):

```
select_por_dni = session.prepare(
    "SELECT * FROM mi_keyspace.usuarios WHERE dni = ?"
)
for row in session.execute(select_por_dni, ("11111111",)):
    print(f"DNI: {row.dni}, Nombre: {row.nombre}, Apellidos: {row.apellidos}, Edad: {row.edad}")
```

Filtrar por `edad` (que no es la clave primaria) exige un índice, como se ve a continuación, o `ALLOW FILTERING` — que recorre todo el clúster y debe evitarse en producción

# Creación de índices SAI

Para crear índices SAI, `CREATE INDEX` junto con `USING 'sai'`:

```
CREATE INDEX apellido_sai_idx ON mi_keyspace.usuarios (apellido)
USING 'sai'
WITH OPTIONS = {'case_sensitive': 'false', 'normalize': 'true', 'ascii': 'true'};
```

- `case_sensitive: false`: las búsquedas no distinguen mayúsculas/minúsculas
- `normalize: true`: normalización Unicode durante las búsquedas
- `ascii: true`: convierte caracteres especiales a su equivalente ASCII

Más opciones: create-custom-index

# Construcción de índices en segundo plano

- `execute_async()` sobre un `CREATE INDEX` sólo confirma que el cambio de esquema **se propagó** por el clúster, no que el índice ya esté construido
- El *build* —recorrer las SSTables existentes y poblar el índice— es un proceso aparte, en segundo plano
- Con varios índices construyéndose a la vez sobre tablas de tamaños distintos, un `time.sleep(N)` fijo puede terminar antes de que estén disponibles: una consulta que use el índice fallaría con `IndexNotAvailableException`
- Mejor: sondear `system_views.sstable_tasks` (equivalente CQL de `nodetool compactionstats`, desde Cassandra 4.0) hasta que no queden tareas de tipo *Secondary index build*, y confirmar con una consulta real de bajo coste

# Construcción de índices: sondeo

```python
import time

def build_en_marcha(session, keyspace: str) -> bool | None:
    """None si system_views no está disponible (plan B: reintentar la query real)."""
    try:
        filas = session.execute(
            "SELECT kind FROM system_views.sstable_tasks WHERE keyspace_name = %s",
            (keyspace,),
        )
    except Exception:
        return None
    return any("index" in (fila.kind or "").lower() for fila in filas)

while build_en_marcha(session, "mi_keyspace"):
    time.sleep(2)
```

# Actualización y eliminación de datos

```
update_query = session.prepare("""
    UPDATE mi_keyspace.usuarios SET edad = ? WHERE dni = ? AND apellidos = ?
""")
session.execute(update_query, (35, '11111111', 'Sanchez'))

delete_query = session.prepare("""
    DELETE FROM mi_keyspace.usuarios WHERE dni = ? AND apellidos = ?
""")
session.execute(delete_query, ('22222222', 'Terroso'))
```

Al terminar, es importante cerrar la conexión con la base de datos:

```
session.shutdown()
```

# Casos de uso y conclusiones

- Entre las limitaciones de Apache Cassandra: no contiene funcionalidad para realizar agregaciones — si se necesitan datos agregados, hay que precomputarlos
- La ventaja principal es su velocidad de escritura y la forma de distribuir los datos, que permite alta escalabilidad
- Para sistemas donde la velocidad de lectura sea crítica, no resulta una solución óptima
- Usos más comunes:
  - **Aplicaciones IoT**: miles de sensores enviando información que debe sincronizarse y correlacionarse en tiempo real
  - **Monitorización de aplicaciones web**: almacenar la actividad de los usuarios en sitios web

# Referencias


- Jim R. Wilson, Eric Redmond. **Seven Databases in Seven Weeks**
- Russell Bradberry, Eric Lubow. **Practical Cassandra — A Developer's Approach**